MAEG3080 Fundamentals of Machine Intelligence

Final Project Report

CIFAR-100 Image Classification with a Custom PyTorch Network

Rex Yim (Yim Fu Chong)
1155176974@link.cuhk.edu.hk

Jason Chen (Chen Jiashen)
115552368890@link.cuhk.edu.hk

The Chinese University of Hong Kong

March 25, 2026

Abstract

CIFAR-100 from scratch (compact SE-ResNet): Stage A → Stage B (**65.17%** test) → Stage C with Mixup/CutMix/EMA (**68.14%** test). Evaluation uses `results.py`.
Public repository: https://github.com/Rex-Yim/CIFAR100Classification.

1 Introduction

We train on CIFAR-100 without pretrained weights (Colab; checkpoints on Drive) and report test accuracy via the course `results.py` script.

Code and repository. All implementation code, shell pipelines, Colab helpers, and LaTeX sources for this report and the companion slides are maintained in the public GitHub repository https://github.com/Rex-Yim/CIFAR100Classification (default branch `main`).

2 Dataset

CIFAR-100: 50,000 train / 10,000 test. Per-channel normalization uses $\mu = (0.5071, 0.4867, 0.4408)$ and $\sigma = (0.2675, 0.2565, 0.2761)$. Stages B/C hold out 10% stratified validation (seed 42); Stage A uses all training images and evaluates on the test set each epoch.

3 Methodology

3.1 Model Architecture

SE-ResNet (`se_resnet`): SE attention, `base_width` = 32, two blocks per stage (2×2×2×2), GAP, linear to 100 classes.

3.2 Training Strategy

- **Stage A:** 4 epochs, basic aug, SGD lr 0.1, warmup 1, cosine; test each epoch; ∼33% checkpoint for init.
- **Stage B:** `--init-from`; RandAugment (N=2, M=9), erasing 0.05, label smooth 0.05, SGD lr 0.05, warmup 5, 30 epochs; val for selection; test via `results.py`. No EMA.

- **Stage C:** From Stage B `best.pt`, 30 epochs (`scripts/iterate_from_stageb_stage_c_mixema.sh`), RandAugment, Mixup/CutMix, lr 0.015, warmup 3, batch 128, clip 1, EMA 0.999. Select by val accuracy.

Code: `train.py`, `model.py`, `utils.py`; Stages A–B via `scripts/quick_se_resnet_results.sh`; Stage C via Colab/`scripts/iterate_from_stageb_stage_c_mixema.sh`. AMP on CUDA. Best test from `results.py` on Stage C `best.pt`.

4 Experiments

4.1 Experimental Setup

Table 1 summarizes Stages A–C; Table 2 reports accuracy.

Table 1: Hyperparameters for the SE-ResNet pipeline (Stages A–B) and Stage C improvement.

Setting	Stage A	Stage B	Stage C
Epochs	4	30	30
Batch size	64	256	128
Augmentation	Basic (crop, flip)	RandAugment + erasing	RandAugment + erasing
Mixup / CutMix	None	None	Both (α 0.2 / 1.0, p=0.5)
EMA	No	No	Yes (0.999)
Optimizer	SGD (momentum 0.9, WD 5×10^{-4})	Same	Same
LR schedule	Warmup 1 + cosine	Warmup 5 + cosine	Warmup 3 + cosine
Initial LR	0.1	0.05	0.015
Label smoothing	0	0.05	0.05
Eval during training	Test (10,000)	Val (10% of train)	Val (10% of train)

4.2 Results

Table 2: CIFAR-100 accuracy. Official test numbers are from `results.py` on the listed checkpoint.

Stage	Metric	Value
A: baseline (4 epochs)	Test accuracy	$\approx$ 33.3%
B: finetune (30 epochs)	Best validation	65.22%
B: finetune	Official test (Stage B `best.pt`)	65.17%
C: from Stage B (30 epochs)	Best validation	68.48%
C: from Stage B	**Official test** (Stage C `best.pt`)	**68.14%**

Stage B validation and test agree ($\sim$65.2%). Stage C reaches best validation 68.48% (epochs 29–30) and **68.14%** official test, indicating that Mixup/CutMix and EMA help when starting from the Stage B weights.

4.3 Training curves and log summary

Figure 1 summarizes per-epoch behavior for Stages A–C. Stages A–B follow the run in Table 2: Stage A on the official test each epoch; Stage B on train and validation (90%/10% split). Stage C plots train and validation under Mixup/CutMix and EMA—**train** accuracy here is not

directly comparable to clean accuracy; use the validation curve and Table 3 (right) for Stage C selection.

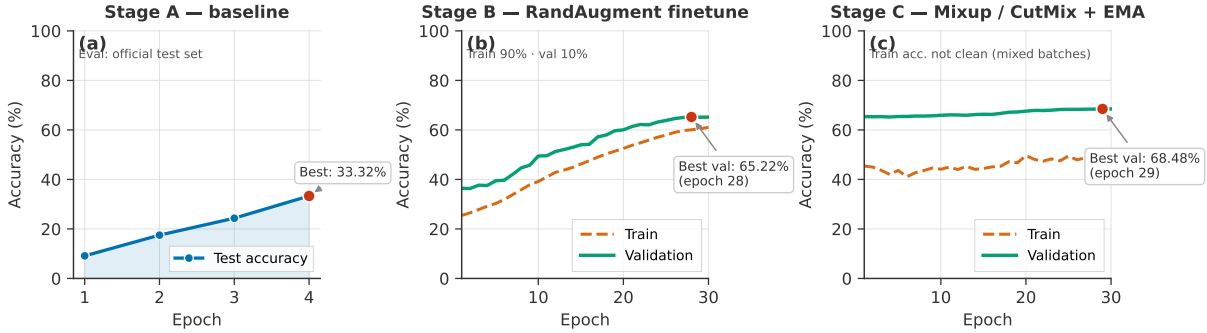


Figure 1: Per-epoch accuracy for Stages A–C (shared y-axis 0%–100%). *Left:* Stage A, official test. *Center:* Stage B, train vs. validation. *Right:* Stage C, train vs. validation (mixed batches). Logs: `artifacts/colab_training_log_4epoch_plus_30_to_65pct.txt` (Stages A–B), `artifacts/stage_c_training_history.json` (Stage C). Script: `scripts/plot_training_curves.py`.

Epoch logs (Stages A–C). Subtables (a)–(c) in Table 3 list selected epochs (full JSON for Stages A–B in the Colab artifact; Stage C in `artifacts/stage_c_training_history.json`). Stage B best validation 65.22% at epoch 28. Stage C best val 68.48% (epochs 29–30); official test **68.14%** via `results.py`.

(a) Stage A (baseline): test accuracy per epoch.

Epoch	Test
1	9.12%
2	17.50%
3	24.33%
4	33.32%

(b) Stage B (finetune): selected epochs. Best val 65.22% at epoch 28.

Ep.	Train	Val
1	25.44%	36.40%
5	30.29%	39.46%
10	39.18%	49.46%
15	46.24%	53.98%
20	52.52%	60.02%
25	57.93%	63.80%
28	60.06%	65.22%
30	61.06%	65.16%

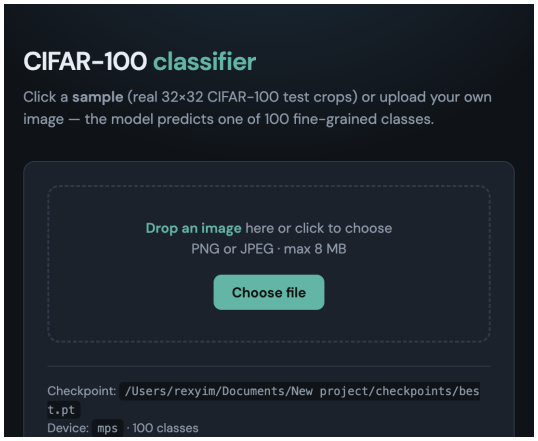
(c) Stage C (`sota_se_resnet_exp1`): validation accuracy.

Ep.	Val
1	65.38%
10	65.90%
20	67.58%
25	68.32%
28	68.40%
29	68.48%
30	68.48%

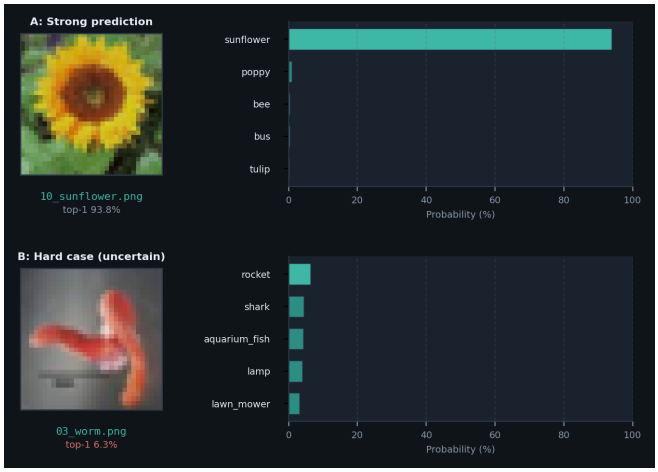
Table 3: Per-epoch training logs for Stages A–C (subtables left to right).

4.4 Interactive web demo

We ship a small FastAPI app under `demo/` (`demo/app.py`): a browser UI to upload an image or use shipped CIFAR-100 32×32 crops, then view top-k softmax probabilities for the same SE-ResNet checkpoint as in evaluation. It runs **locally** (e.g. `uvicorn demo.app:app`) and is for qualitative inspection only; official numbers still come from `results.py`.



(a) Browser UI (local server).



(b) Two crops from `demo/static/samples/`: highest vs. lowest top-1 confidence among shipped PNGs (generated with `scripts/render_demo_slide_panels.py`).

Figure 2: Optional local demo for qualitative behavior (not used for grading metrics).

5 Discussion

- **Initialization matters:** Starting from the Stage A weights gave a large jump over training the finetune recipe from scratch in the same wall-clock budget.
- **RandAugment + longer schedule:** Stronger augmentation and more epochs on top of the Stage A baseline outperformed a larger Wide-ResNet trained from scratch in the same time budget.
- **Stage C on top of Stage B:** Treating the ∼65% model as a *base* checkpoint and finetuning with Mixup, CutMix, and EMA at a lower learning rate yielded ∼3 percentage points on official test (65.17% → 68.14%), with best validation 68.48%.
- **Trade-offs:** The SE-ResNet path is faster to iterate than a full Wide-ResNet Hydra ladder; the cost is a lower ceiling than very large models trained for hundreds of epochs on CIFAR-100.

6 Contribution

Rex Yim led implementation: data pipeline, preprocessing and augmentation experiments, model code, training and checkpoints, analysis, and primary drafting of this report and slides. Early drafting used OpenAI Codex; later work used Cursor. Stages B and C ran on Google Colab (NVIDIA GPU).

Jason Chen contributed brainstorming, discussions on preprocessing and training schedule (including epoch sizes), and support for presentation preparation and Q&A practice.

7 Conclusion

We trained a compact SE-ResNet on CIFAR-100 (Stages A–B), then improved it with Stage C (Mixup, CutMix, EMA) from the Stage B checkpoint. The best official test accuracy is **68.14%** (`results.py`), with Stage B alone at **65.17%**. Future work could use longer schedules or wider models if compute permits.

References

1. A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” 2009.
2. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” 2016.
3. J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” 2018.
4. TorchVision and PyTorch official documentation.